

RRCLite Program Architecture Analysis

1. Project Architecture Diagram

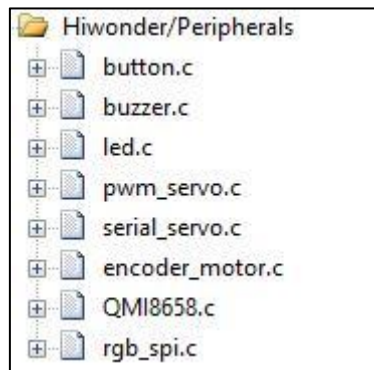
The project structure analysis diagram is a functional architecture diagram of the controller. It outlines the main source code files that the controller will use. The contents of the files involved are located in the "Appendix/Factory Examples" folder.

2. Architecture Analysis

The RRCLite program mainly consists of four main parts: Peripherals, Chassis, Porting, and System.

Next, each part will be explained.

2.1 Peripherals



Peripherals involves the software initialization and execution functions of the relevant components on the STM32 main control board, including:

Buzzer (buzzer.c): This file is used to implement the control for the hardware and system-independent buzzer. The class concept abstracts the functions such as buzzer switching, cycle sounding and initialization in these functions. The called task function in rtos timer callback function related with the buzzer is defined.

Button (button.c): This file is used to implement hardware-independent button detection. The callback function for button presses is defined here. It

enables implementing button detection functionality across different hardware platforms using the same code, requiring only a specific implementation for reading button states. It also supports various button events.

IMU (QMI8658.c): This file is for achieving the hardware-independent IMU data calculating function, as well as the hardware-related read and write function.

Encoder DC Motor Interface (encoder_motor.c): This file enables encoder motor hardware-independent code. The functions such as PID calculating current motor PWM value and measuring speed are defined.

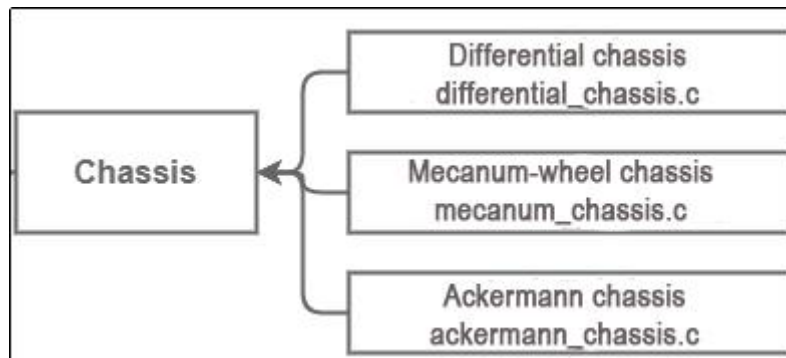
RGB (rgb_spi.c): This file allows RGB light hardware and system-independent code. Only SPI protocol sending relevant data frame can make MCU control RGB light. In this c file, the data frame format, in the array format, passed by MCU threading out through SPI interface are defined. The relevant initial function, as well as implementation functions that control the current RGB light and all RGB lights are provided.

LED (led.c): This file contains hardware- and system-independent code for LED functionality. It defines parameters, functions, and tasks related to LED operation.

PWM Interface (pwm_servo.c): This file contains hardware-independent code for PWM servo control. It provides an interface to control PWM servos, allowing them to move to specific positions within specified time frames and enabling deviation correction.

Bus Servo Interface (serial_servo.c): This file is for bus servo driving and hardware-independent code. The communication between MCU and the bus servo mainly depends on the definition and writing of the data frame. In this program, define the data frame structure as a structure body. In this c file, it provides the data frame writing function controlling multiple bus servo commands such as reading ID and setting ID, and initialization function.

2.2 Chassis



The Chassis folder contains implementations of kinematic algorithms for different types of chassis (velocity, acceleration, turning radius), including:

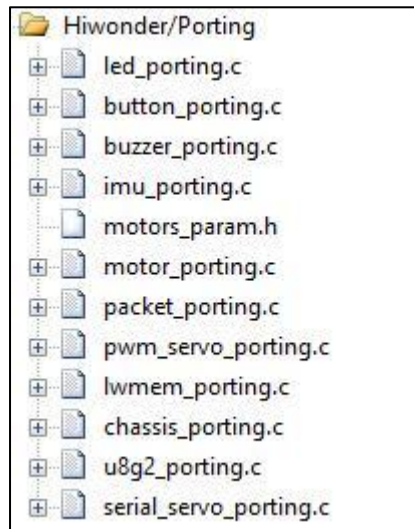
Differential Chassis (differential_chassis.c): Implementation of kinematic code for a differential drive chassis. The main functions to focus on are “diff_chassis_move” and “set_velocity_radius”.

Mecanum Chassis (mecanum_chassis.c): Implementation of kinematic code for a mecanum drive chassis. The main functions to focus on are “mecanum_chassis_set_velocity”, “mecanum_chassis_set_xy”, and “set_velocity_radius”.

Ackermann Chassis (ackermann_chassis.c): Implementation of kinematic code for an Ackermann drive chassis. The main functions to focus on are “ackermann_velocity_difference”, “ackermann_chassis_move”, and “set_velocity_radius”.

2.3 Porting

Porting involves the implementation functions for the interfaces of components on the STM32 main control board and related sensor accessories that can be mounted. When calling these functions later on, particular attention can be paid to the "hiwonder/Porting" folder.



LED (led_porting.c): Example and interface for controlling onboard LED lights.

The main functions to focus on are:

leds_init: Used to initialize LED-related memory and variables.

led_set_pin: Used to set the state of the LED lights (on/off).

Button (button_porting.c): Example and interface for controlling onboard buttons. The main functions to focus on are:

buttons_init: Used to initialize button-related memory and variables.

button_timer_callback: This is a timer callback function used for periodically scanning the state of the buttons.

Buzzer (buzzer_porting.c): Example and interface for controlling the onboard buzzer. The main functions to focus on are:

buzzers_init: Used to initialize buzzer-related memory and variables.

buzzer1_set_pwm: Buzzer PWM setting interface used to set the PWM frequency of the buzzer.

Odometer (imu_porting.c): Interface file for QMI8658, retrieving the three-axis angular values of the IMU odometer. The main functions to focus on are:

imus_init: Used to initialize the IIC address of QMI8658 and related parameters.

imu_task_entry: IMU task entry function. In the loop, it waits for a semaphore,

then updates the IMU data, retrieves acceleration and gyroscope data, and sends these data packets.

Bus Servo (serial_servo_porting.c): The main functions to focus on are:

`serial_write_and_read`: The underlying implementation for MCU to receive and send data of bus servo.

`serial_servo_init`: Configures serial ports related to receiving and sending data.

Motor Driver (motor_porting.c): Used to drive the motors for rotation.

The main functions to focus on are:

`motors_init`: Initialization function for the motors. Used to initialize parameters such as PID parameters and rotation speed limits.

`set_motor_param`: Writing PID parameters for the motors.

PWM (pwm_servo_porting.c): Used for controlling PWM servos.

The main functions to focus on are:

`pwm_servos_init`: Initialization function for the PWM interface. This function initializes four PWM servo objects and configures a timer to generate PWM signals. Each servo object has a corresponding write function associated with it. These functions send PWM signals to the physical pins of the servo, thereby controlling its rotation.

`pwm_servo1_write_pin`: Control function for PWM servo 1.

Packet protocol (packet_porting.c): Used to calculate various specified parameter data and events from the data received from the serial port or configure them into the files of data frames to be sent.

`packet_tx_task_entry`: task entry for sending data frames.

`packet_rx_task_entry`: task entry for receiving data frames.

Interface Porting and Memory Space Definition (lwmem_porting.c):

Definition of memory space addresses.

`lwmem_region_t` structure array: Each structure defines a memory region. The first area starts at address 0x10000000 with a size of 64KB, used for RAM on the microcontroller. The second area is the `lwmem_ram`, defined earlier, with a

size of 38KB. The last item {NULL, 0} serves as the end marker for the array.

Chassis Control Interface (chassis_porting.c): Used for controlling chassis movement. It defines objects for five different types of mobile chassis, provided for user selection. When using it, users can choose the appropriate chassis for control.

```
DifferentialChassisTypeDef jetank;  
DifferentialChassisTypeDef tank_black;  
DifferentialChassisTypeDef ti4wd;  
MecanumChassisTypeDef    jetauto;  
AckermannChassisTypeDef  jetacker;
```

chassis_init: Initializes the chassis.

set_chassis_type: Selects the type of chassis.

u8g2 (u8g2_porting.c): Used for driving OLED screens with the u8g2 library. It serves as the OLED screen driver interface.

u8g2_init: Initializes the u8g2 object, registers relevant driver interfaces, and initializes the screen.

u8x8_gpio_and_delay: This function provides GPIO and delay operations for the u8g2 library.

u8x8_byte_hw_i2c: This function is a hardware-level interface of the u8g2 library used for implementing I2C communication with the OLED screen.

2.4 System

The "System" folder contains system files responsible for initiating various tasks and other component-related initializations. It is mainly divided into four parts: battery detection, serial port data packet processing, OLED display, and main application logic.



Battery Detection (battery_handle.c): Used for detecting battery voltage, calculating actual voltage, and sending low voltage alerts.

battery_check_timer_callback function: Key aspects include how to handle and report battery voltage, as well as how to respond to low voltage situations. This function covers several key functionalities: 1. Reading and filtering battery voltage. 2. Periodically sending battery voltage to GUI or Bluetooth. 3. Low battery alerts.

OLED Display (gui_handle.c): Used for updating data, information, and display content in the OLED interface.

This development board does not specifically reserve a hardware interface for the OLED display screen. However, there is an I2C interface on the board. If necessary, please refer to this lesson for secondary development.

Main Application Logic (app.c): The startup file for the main application program.

This typically contains major task entry functions, tasks created in FreeRTOS, and message queue tasks. This program describes the main application logic of a robot, involving hardware initialization, configuration, and response to button events.

The operations can be implemented within the main function or the entry function "app_task_entry" of the user task in this file.

Serial Port Data Packet Processing (Packet_handle.c): Provides data frame structures for various events and parameters. Defines the data frame encapsulation method for all parameters and event transmission.

Note: Before making any modifications to the program, be sure to back up the original factory program. Avoid making direct modifications to the source code files to prevent any accidental parameter changes that may lead to robot malfunctions and become irreparable!!!
